1 Introduction

This is a shortened version of the MPI standard, containing only the sections relevant to the test suite. It is not intended to be a complete reference for MPI.

2 Terms and Conventions

3 Point-to-Point Communication

This chapter contains some of the most commonly used MPI functions, such as `MPI_Send` and `MPI_Recv`. The full list of point-to-point communication functions can be found in the official MPI standard.

The MPI function `MPI_Send` is used to send a message from one process to another. It takes the following parameters:

```
int MPI_Send(const void* buf, int count, MPI_Datatype datatype,
int dest, int tag, MPI_Comm comm)
```

`buf` is a pointer to the data to be sent, `count` is the number of elements in the data, `datatype` is the type of the data, `dest` is the rank of the destination process, `tag` is an integer tag to identify the message, and `comm` is the communicator.

The non-blocking version of this function is `MPI_Isend`, which allows the sender to continue executing while the message is being sent.

```
int MPI_Isend(const void* buf, int count, MPI_Datatype datatype,
int dest, int tag, MPI_Comm comm, MPI_Request* request)
```

`buf` is a pointer to the data to be sent, `count` is the number of elements in the data, `datatype` is the type of the data, `dest` is the rank of the destination process, `tag` is an integer tag to identify the message, `comm` is the communicator, and `request` is a handle to the request object. The sender can use the request object to check the status of the non-blocking send operation.

The MPI function `MPI_Recv` is used to receive a message from another process. It takes the following parameters:

```
int MPI_Recv(void* buf, int count, MPI_Datatype datatype,
int source, int tag, MPI_Comm comm, MPI_Status* status)
```

`buf` is a pointer to the buffer where the received data will be stored, `count` is the maximum number of elements that can be received, `datatype` is the type of the data, `source` is the rank of the source process, `tag` is an integer tag to identify the message, `comm` is the communicator, and `status` is a pointer to a status object that will be filled with information about the received message. The receiver can use the status object to check the source, tag, and error code of the received message.

4 Datatypes

5 Collective Communication

The simplest collective communication function is `MPI_Barrier`, which blocks until all processes in the communicator have reached the barrier. It takes the following parameters:

```
int MPI_Barrier(MPI_Comm comm)
```

`comm` is the communicator that defines the group of processes that must reach the barrier before any of them can proceed.

Another collective communication function is `MPI_Bcast`, which broadcasts a message from one process to all other processes in the communicator. It takes the following parameters:

```
int MPI_Bcast(void* buffer, int count, MPI_Datatype datatype,
int root, MPI_Comm comm)
```

`buffer` is a pointer to the data to be broadcasted, `count` is the number of elements in the data, `datatype` is the type of the data, `root` is the rank of the root process that initiates the broadcast, and `comm` is the communicator.

Sending different data to different processes can be achieved using the `MPI_Scatter` function, which distributes distinct pieces of data from one process to all other processes in the communicator. It takes the following parameters:

```
int MPI_Scatter(const void* sendbuf,
int sendcount, MPI_Datatype sendtype,
void* recvbuf, int recvcount, MPI_Datatype recvtype,
int root, MPI_Comm comm)
```

`sendbuf` is a pointer to the data to be sent, `sendcount` is the number of elements to send to each process, `sendtype` is the type of the data to send, `recvbuf` is a pointer to the buffer where the received data will be stored, `recvcount` is the number of elements to receive, `recvtype` is the type of the received data, `root` is the rank of the root process that initiates the scatter, and `comm` is the communicator.

If the messages being sent to different processes have different sizes, the `MPI_Scatterv` function can be used instead, which allows for varying counts and displacements for each process. It takes the following parameters:

```
int MPI_Scatterv(const void* sendbuf,
const int* sendcounts, const int* displs,
MPI_Datatype sendtype,
void* recvbuf, int recvcount,
MPI_Datatype recvtype, int root, MPI_Comm comm)
```

`sendbuf` is a pointer to the data to be sent, `sendcounts` is an array specifying the number of elements to send to each process, `displs` is an array specifying the displacement (offset) of the data to send to each process, `sendtype` is the type of the data to send, `recvbuf` is a pointer to the buffer where the received data will be stored, `recvcount` is the number of elements to receive, `recvtype` is the type of the received data, `root` is the rank of the root process that initiates the scatter, and `comm` is the communicator.

the following definition should not be included in the list of MPI functions.

The `copy_fn` function is invoked when a communicator is duplicated by `MPI_COMM_DUP`. `copy_fn` should be of type `MPI_Copy_function`, which is defined as follows:

```
typedef int MPI_Copy_function(MPI_Comm oldcomm, int keyval,
void *extra_state, void *attribute_val_in,
void *attribute_val_out, int *flag)
```